Modular composition of SPARQL queries for focusing on what to look for rather than how to get it

Yael Tirlet Greiner[1], Jerven Bolleman[2], Emmanuelle Becker[1], Fabrice Legeai[3] and Olivier Dameron[1,*]

[1]Univ Rennes, Inria, CNRS, IRISA - UMR 6074, F-35000 Rennes, France
[2]SIB Swiss Institute of Bioinformatics, 1, rue Michel Servet - CH 1211 Geneva 4 - Switzerland
[3]IGEPP, INRAE, Institut Agro, Univ Rennes, 35653, Le Rheu, France

Abstract

Adoption of life science knowledge bases by domain experts remains low in spite of the increasing accessibility of these bases, as the Semantic Web framework supports advanced integration and querying. The main bottleneck for leveraging these knowledge bases is that advanced querying combines the inner complexity of life sciences (which requires domain expertise) with the technical complexity of knowledge bases schemas and of SPARQL (which requires engineering skills).

We propose a framework based on modules that reconciles both views. A module corresponds to a concept relevant to domain experts and is associated with a SPARQL fragment compliant with the data schema. Modules can be connected to compose new modules corresponding to more complex concepts; the SPARQL fragments of the components are automatically combined to constitute the fragment of the composed module. Our approach thus allows experts to focus on what they are looking for, while our system takes care of how to obtain it.

Keywords

Semantic Web, SPARQL, Query composition

1. Introduction

Many biology databases are now accessible *via* Semantic Web technologies [1, 2, 3, 4, 5]: RDF for data representation, OWL for knowledge modeling, and SPARQL for unified resource querying [6, 7, 8]. These standards make it possible to (i) formulate expressive queries on heterogeneous data backed by a knowledge base, (ii) link information from multiple sources, and (iii) compose federated queries spanning multiple databases. However, in addition to the complexity of the SPARQL language itself [9], the first of these points raises a major difficulty: the complexity of the underlying schema. This complexity results both from the intrinsic richness of the biological domain and from the technical constraints associated with data modeling and implementation. This complexity becomes even more pronounced when it comes to reasoning across multiple databases or exploiting interconnections between ontologies, thus constituting a significant barrier to the adoption of these resources by life science experts [10, 11].

In practice, when domain experts want to answer a domain question, it is difficult for them to query a SPARQL endpoint directly [12]. Proficiency in the language is not common among life scientists, and even when query examples are provided, they are often unclear: the link between the functional description of a need and the concrete structure of the query remains opaque. Data schemas are sometimes simplified through manual diagrams or supplemented by query libraries (such as those designed for UniProt [13, 12]), but these solutions are not enough to overcome the technical barrier. The main challenge remains how to help experts combine familiar concepts and automatically match them to appropriate SPARQL patterns.

SWAT4HCLS 2026: Bridging Life Sciences and Technology, March 23–26, 2026, Amsterdam
*Corresponding author.
yael.tirlet@irisa.fr (Y. Tirlet Greiner); jerven.bolleman@sib.swiss (J. Bolleman); emmanuelle.becker@irisa.fr (E. Becker); fabrice.legeai@inrae.fr (F. Legeai); olivier.dameron@irisa.fr (O. Dameron)
0009-0008-3713-2142 (Y. Tirlet Greiner); 0000-0002-7449-1266 (J. Bolleman); 0000-0002-7673-0582 (E. Becker); 0000-0002-6472-4839 (F. Legeai); 0000-0001-8959-7189 (O. Dameron)
© 2026 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

Our contribution is a framework for coupling a functional view on a knowledge base (that focuses on *what* information it contains) with its data schema (that defines *how* this information is represented). This functional view is organized into modules, each corresponding to a concept relevant to domain experts and associated with an underlying SPARQL fragment compliant with the data schema. Modules can be connected to compose new modules corresponding to more complex concepts; the SPARQL fragments of the components are automatically combined to constitute the fragment of the composed module. Our approach thus offers a simple and intuitive way to access the expressive potential of SPARQL: experts focus on what they are looking for, while our system takes care of how to obtain it.

2. Background

Various approaches have been proposed for making querying complex life science knowledge bases exposed as SPARQL endpoints accessible to end users. We distinguish those targeting the complexity of the SPARQL language from those targeting the complexity of the data schema.

For allowing non-SPARQL literate users to design queries, approaches encompass simplifying the identifiers and variables manipulated by SPARQL, simplifying SPARQL itself and interactive exploration of the data schema. SPARQL Assists [14] helps query composition by presenting users with the `rdfs:label` of opaque and semantic-free URI (e.g. "is homologous to" for `sio:010302`), and context-sensitive type-ahead completion of already used variables. It addresses some of the syntactic difficulties of writing SPARQL, but not that of SPARQL itself, nor the technical complexity of the data schema. Another paradigm is to propose a graphical user interface that handles the syntax of the language. Two projects took inspiration from block-based visual programming such as Blockly [15] and Scratch [16]: sparql-blockly[1] and SparqlBlocks[2] [17]. They allow users to assemble visual blocks representing the various elements of a SPARQL query. While users do not have to write SPARQL code in a text file, the cognitive task of assembling the blocks is exactly that of the SPARQL language. In practice, the main difficulty of designing a query lies more in understanding and combining the SPARQL concepts than in representing them as text. In addition, the user still needs to understand the data schema of the knowledge base. SPARQLGraph [18] provides a visual interface for assembling triples by entities drag and drop, with contextual drop-down menus presenting the various predicates applicable to an entity. The visual interface allows to create and share query graphs. It was deployed with knowledge bases from the EBI and bio2rdf SPARQL endpoints. SPARQL Builder [19] allows to search instances in an output class given some instances in an input class. The user specifies the input and output classes, and the system combines a path finder on the data schema and a query constructor. The tool supports visual query composition at the level of the graph of classes. More recently, sparnatural [20] provides the most advanced visualization for composing triple patterns, based on the data schema.

Another line of works focuses on the complexity of the data schema of life science knowledge bases. SPANG [21] is a set of command lines for writing queries that introduces the notion of SPARQL templates. Even if end-users are not likely to craft elaborate command line scripts, knowledge engineer can create some templates that would be easier to manipulate and that can shield users for some of the technical difficulties related to the data schema. These templates are not really documented though, so users still have to read the corresponding SPARQL code to gain some understanding on what they do. Beside manually-created templates, Campinas et al. used graph summarization techniques on the data schema that "automatically generate a more concise representation of a data schema by presenting only important schema elements while achieving broad information coverage" [22]. It generates a simplified view of the data schema including across several data sources, but does not address the of SPARQL itself. PIBAS FedSPARQL [23] also supports the creation of manually-created templates, including across multiple knowledge bases though federated queries. However, these templates cannot be combined.

Overall, three key factors stand out for making life science SPARQL endpoints accessible to life science users without requiring programming skills above scripting: (1) address the technical aspects

[1] https://github.com/langsamu/sparql-blockly
[2] https://sparqlblocks.org/

of writing SPARQL, (2) handle but do not expose the complexity required by knowledge engineering such as reification or named graphs, and (3) allow users to not only reuse query templates, but also to incrementally combine them. None of the current works cover these three aspects.

3. SPARQL Modules

In this section, we present the general principles of our approach (Section 3.1), then we define the components of our framework (Section 3.2), and specify how the SPARQL code for a query is generated when modules are connected (Section 3.3).

3.1. Principle and approach

We propose a framework that reconciles conceptual composition (which relies on domain expertise) with the technical aspect of data representation and querying (which relies on engineering). Our approach is to represent concepts that make sense for domain experts as *modules*, and to associate each module with a fragment of SPARQL query on a knowledge base (Figure 1). Each module provides a mapping between its functional view (the description of the concept that specifies *what* it does) and its technical view (the SPARQL fragment that specifies *how* to do it). Modules are further defined in Section 3.2.1.

Each concept exposes one or more elements that the user can interact with, that we call *ports*. For example, the concept of Person would encompass an identifier for the person, a name, as well as other biographic information. The user may interact with them by retrieving their value or by providing some constraints on their value depending on what the user is interested in. Ports are further defined in Section 3.2.2.

A port of a module can be connected to a port of another module to convey the intention that the ports have the same value. This generates a more complex concept that can also be seen as a module. The new module SPARQL fragment can be determined by applying an operation on the respective SPARQL fragments of the two connected modules and on the variables of the connected ports (e.g. renaming them so that they are the same). A major challenge for this operation is to avoid any unintended collision of variable names. Port connections as well as variable name collision are further defined in Section 3.2.3.

Users may have to manipulate several modules that are identical (which makes the problem of variable collision even worse), e.g., to connect each person of the module "person1 knows person2" with a distinct "person has name" module in order to retrieve the names of the persons who know each-other. In Section 3.2.4, we define a cloning operation for modules, and show that the general variable collision problem can be solved by using a set of modules as templates, and only using their clones for composing queries.

Manipulating modules may involve some advanced operations that have a direct counterpart in SPARQL (e.g., to perform a negation, or union, or to specify that the presence of a module is optional). Our framework defines these operations as special modules, which we call *module operators*. Module operators are further defined in Section 3.2.5.

3.2. Definitions

This section first defines the notions of module and ports, and then the operations allowing to combine modules. The combination of several modules results in a new module. For each operation allowing to combine modules, we define the operation it performs on their SPARQL fragments.

3.2.1. Definition of a Module

We define a *module* in a knowledge base as an explicit formalization of a concept relevant to domain experts. The explicit formalization of this concept is composed of its description conveying

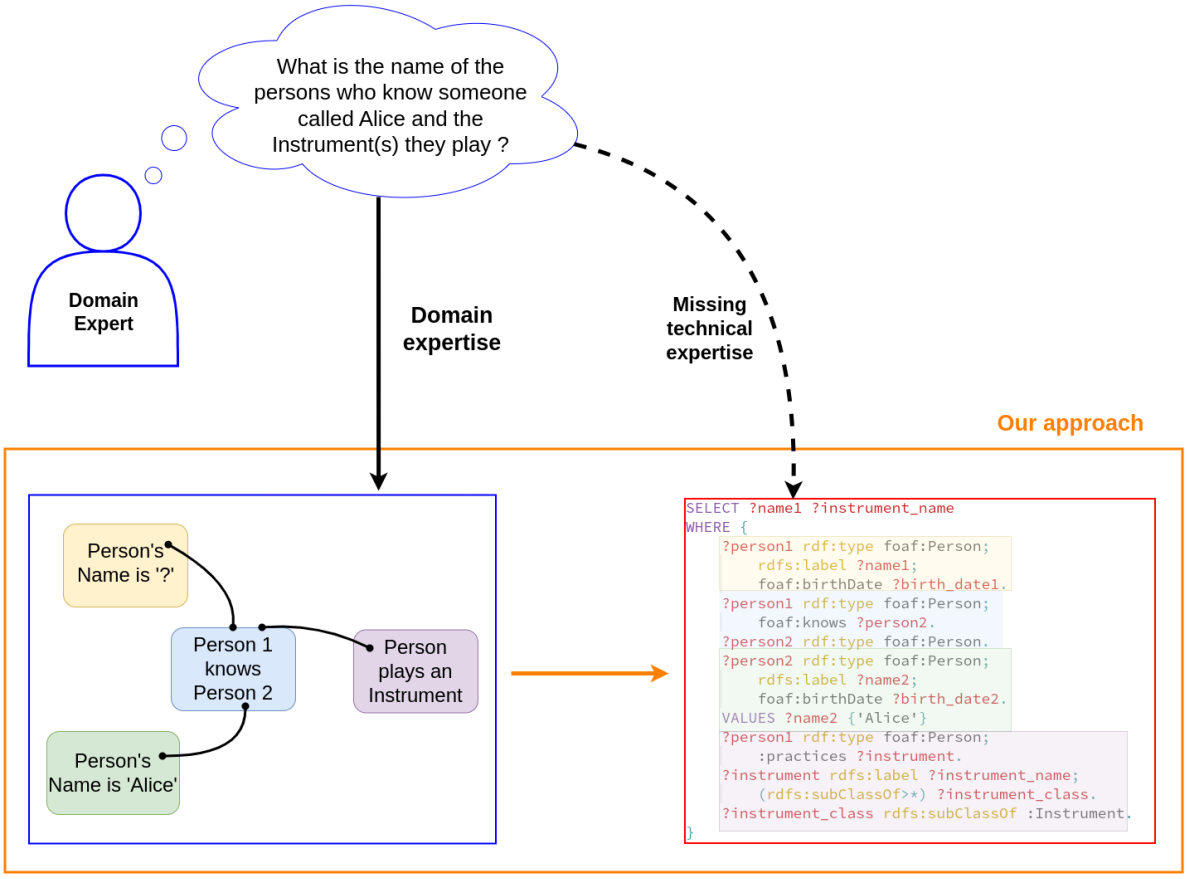

Figure 1: Our modular approach for composing SPARQL queries. We propose a framework based on SPARQL modules. Modules correspond to concepts that make sense for domain experts. They map a functional description that represents *what* the concept means and its possible interactions, with a SPARQL fragment that is a technical specification of *how* the concept is represented according to the knowledge base schema. In the figure, the module descriptions are the four colored boxes in the lower left corner. The respective SPARQL fragments of the modules have the same color as their description in the query in the lower right side. Domain experts interact only with the module descriptions (blue frame) by connecting them, and our system automatically recomposes the query using the corresponding SPARQL fragments (red frame). Connecting two elements means that they represent the same thing (e.g. the person in the yellow module is both the one who knows in the blue module, and the one who plays some instrument in the purple module, whereas the person in the green module is the one who is known in the blue module), which results in using the same variables in the corresponding SPARQL fragments.

what it means, and of a companion SPARQL fragment indicating *how* it is represented (Figure 2).

The concept is a logical combination of characteristics covered by the knowledge base. It is a mental construction in the expert's head. Formulating a question on a knowledge base consists in describing the underlying concept. This description can be communicated. Turning this question into a query consists in transcribing the description based on how information is represented in the knowledge base. The query is processed by a software that retrieves the (possibly empty) set of pieces of information from the knowledge base that match the characteristics of the concept associated to the query. Currently, experts have to articulate their question directly as a query (Figure 1). Modules bridge the gap between the formulation of the question and the design of the query. They reconcile the functional view on a knowledge base provided by the concept descriptions and the structural view provided by its data schema.

The description of a module is composed of one or more elements called ports, that allow the user to interact with the underlying concept. Each of these elements has a counterpart as a variable in the SPARQL fragment of the module. The association between an element of a concept and the

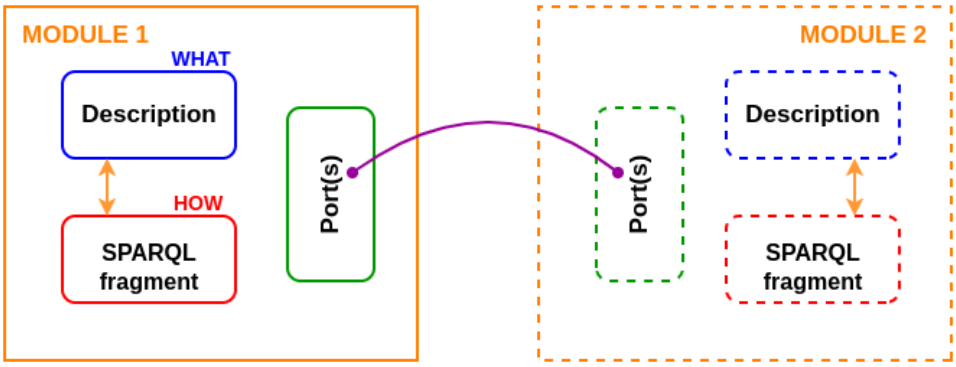

Figure 2: A module reconciles a functional view and a structural representation of a concept, and can be connected to other modules. A module corresponds to a concept about the information stored in a knowledge base. It maps the description of the concept that specifies what the concept means, and the corresponding SPARQL fragment that specifies how the concept is represented in the knowledge base. Modules expose their textual descriptions (blue frame) and their ports (green frame). Users use the description to determine the meaning of a module and to identify the ports corresponding to its main elements. Users interact with modules through their ports for retrieving some elements' values, setting constraints on these values, or connecting them to ports from other modules. Connecting two modules results in a more complex module.

corresponding SPARQL variable is further defined in Section 3.2.2.

The SPARQL fragment defines how the connections between the variables of the ports are represented according to the knowledge base schema. This SPARQL fragment is generally a Basic Graph Pattern (BGP), i.e. a set of triples patterns which components are IRIs, datatype values or variables, but it can also contain more elaborate components such as FILTER NOT EXISTS, OPTIONAL,... (c.f. operators in Section 3.2.5) or sub-queries, or a SERVICE graph pattern for federated queries. Although not formally required, one would expect the triple patterns of the SPARQL fragment to form a connected graph.

A module is composed of: an identifier, a textual description, a SPARQL fragment, a set of prefix declarations that cover all the prefixes used in the SPARQL fragment and a non-empty set of ports for which the corresponding variables are all present in its SPARQL fragment and such as all the ports are associated to different variables.

3.2.2. Definition of a Port

We define a *port* as an element that allows users to interact with a module. As such, it is a component of the module's concept description, and it is associated to one of the variables of the module's SPARQL fragment. For example, the ports of the concept "a person and her biographic information" could be an identifier for the person as well as a name, a birthdate, etc. As users interact with modules through their ports, a module must have at least one port. Each port of a module is associated to exactly one variable in the SPARQL fragment of the module. Each variable in the SPARQL fragment is associated to at most one port.

The user interactions with a port encompass:

- retrieving the element's value (e.g. the name or the URI of the person in the orange module in Figure 1, cf. Section 3.3.1),
- or setting constraints on its value:
  - either by setting it by enumeration (e.g. setting the value of the name to "Alice" in the green module, see operator VALUES in Section 3.2.5), or by applying some conditions with FILTER,
  - or by connecting it to another port, which states that the values of the two ports are the same (cf. Section 3.2.3).

Note that it may not be desirable to allow users to connect a port of a module to just any port of any other module (e.g., to prevent them from connecting the port representing the identifier of a person to

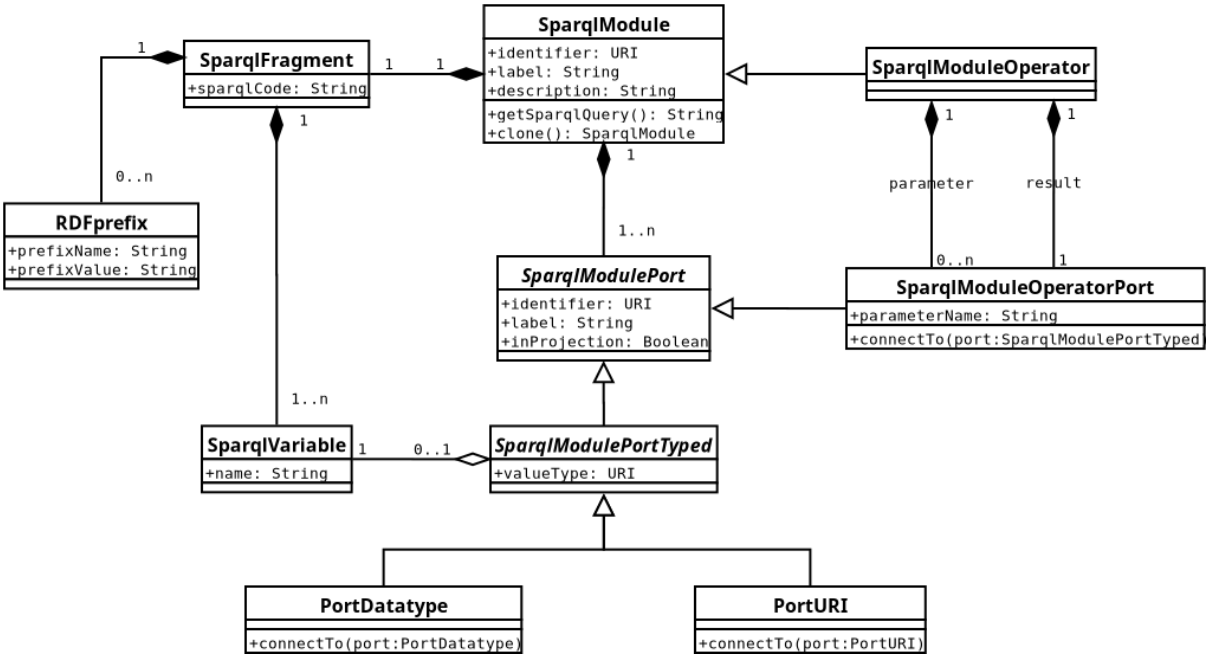

Figure 3: Class diagram for modules, ports and variables. A module is composed of a SPARQL fragment that contains SPARQL variables, and of one or more ports. Each port is associated to a distinct variable of its module's SPARQL fragment. Depending on the nature of the variable values, we distinguish datatype ports and URI ports, that can only be connected to ports of the same kind. Module operators are a special kind of modules that perform operation on the modules their parameters are connected to.

a port representing a birthdate or to a port representing the identifier of a musical instrument). The remainder of this section focuses on how to determine whether two ports can be connected. Section 3.2.3 defines what happens to the modules which have (consistently) connected ports. For ensuring that port connections are consistent with domain knowledge, module ports are both syntactically and semantically typed. On the syntactic level, we distinguish *Datatype ports*, i.e. ports associated to a variable which value belongs to a datatype, from *URI ports*, i.e. ports associated to a variable which value is an URI. A datatype port can only be connected to another datatype port, and an URI port can only be connected to another URI port (Figure 3). On the semantic level, each port can be associated to a type. For datatype ports, this allows to represent (typically using the XSD vocabulary[3]) how the value should be interpreted (default is `xsd:string`). This prevents from connecting a datatype port representing a birthdate (typed with `xsd:date`) with another datatype port representing a living status (typed with `xsd:boolean`). For URI ports, this allows to represent the nature of the element identified by the URI by an RDFS or OWL class (default is `xsd:anyURI`). This prevents from connecting an URI port representing a `foaf:Person` to an URI port representing a `ex:MusicInstrument`, but allows to connect it to an URI port representing a `foaf:Person`, or to an URI port representing a `ex:Musician` provided that the knowledge base specifies that `ex:Musician` is a subclass of `foaf:Person`. Such consistency checks are not mandatory and would typically be performed by the applications that create or manipulate modules. Note that the SPARQL fragment associated to a module may contain type constraints on the variable associated to a port, and that this is independent from the port typing mechanism described above: in the SPARQL fragment, the knowledge engineer may choose to not assign any explicit type to a port variable, or to use a flat set of classes using triple patterns such as:

```
?someVariable rdf:type ex:SomeClass .
```

or to handle class hierarchy using triple patterns such as:

```
?someVariable rdf:type/(rdfs:subClassOf*) ex:SomeClass .
```

[3]https://www.w3.org/TR/xmlschema11-1/

A port is either a datatype port or an URI port. It is composed of: an identifier, a textual label, a value type which is typically an URI from the XSD vocabulary for datatype ports (default is `xsd:string`), or the URI of a class for URI ports (default is `xsd:anyURI`, a name of a variable from the SPARQL fragment of its module, and a boolean indicating whether the user wants to retrieve its values when composing a SPARQL query.

3.2.3. Definition of a connection between ports

As mentioned previously, a port can be connected to 0, 1 or several other ports. Connected ports convey the intention that the elements they represent are the same. Therefore, connected ports have to be both syntactically and semantically compatible (cf. Section 3.2.2).

Connecting two ports consists in:

1. Merging the two ports
    - Choose a common variable name (either the name of one of the two variables, or a new unique one) and rename the two variables in their modules' SPARQL fragments. Renaming the variable of a port consists in updating the variable name for the port, and replacing all the occurrences of the former name by the new name in the SPARQL fragment of the port's module.
    - Choose the most general of the two ports' value types. As the two ports are semantically-compatible, either they have the same value type, or one the two value types is a transitive `rdfs:subClassOf` of the the other. In the first case, the value type of the merged port is the common value type. In the second case, it is the transitive superclass.
    - Determine whether the user wants to retrieve the port's values when composing a SPARQL query by performing a logical OR on each connected port status.
2. Determining the module resulting from the connection:
    - Each of the two connected ports is replaced by the merged port.
    - If the two ports belong to the same module then the module resulting from the connection is simply the original module after having renamed the variables and merged the ports.
    - If the two ports belong to different modules, then the module resulting from the connection is a new module. After having renamed the variables and merged the ports, the new module's SPARQL fragment is the concatenation of the fragments of the two original modules (Figure 4), its set of ports is the union of their sets of ports, and its set of prefix declarations is the union of their sets of prefixes.

If connecting ports from different modules, it is important that the variables of their two SPARQL fragments have different names (except for the variable of the connected ports) in order to prevent unexpected name collisions. This can be achieved either at runtime by automatic query rewriting, or beforehand during module creation. Module cloning presented in section 3.2.4 offers a solution for generating modules with different variable names. After merging the connected ports, all the ports from both modules are associated to variables with different names, which prevents collision when performing the union of both modules ports, and guarantees that each SPARQL variable of the result is still associated to at most one port.

Note that two modules can be connected by more than one pair of ports (e.g. to represent the pairs of persons having the same name and the same birthdate). In this situation, merging the ports of one of the pairs (e.g. the names) results in the creation of a new module that contains the (still connected) ports of the other pair. We are then in the situation of two connected ports from the same module, and simply merge their ports.

3.2.4. Module cloning and a solution for avoiding variable collisions

When composing a query, it may be necessary to use several instances of the same module. For example, retrieving the persons who know both Alice and Charles requires three instances of the "Person" module:

one for Alice, one for Charles, and one for the persons who know them. This requires a mechanism that both facilitates duplication and prevents possible collision of the variables in their respective generated SPARQL fragments.

We define *module cloning* as creating a new module from an existing module. Compared to the original module, the new module has a different identifier, the same description, the same SPARQL fragment after having systematically renamed all the variables, and its set of ports is composed of the clones of the original module ports, adjusted to the variable renaming. Similarly, cloning a port consists in creating a new port with the same characteristics as the original, but with a different identifier and a different variable name.

Module cloning can also be used to solve the more general problem of avoiding variable collision when connecting two modules. The approach consists in using a pool of modules as templates. This pool can evolve, and some modules of the pool *can* even have variables having the same name. The modules of the pool *must not* be used directly for composing queries. For composing queries, the user *only* uses clones of modules from the pool (and a module from the pool can be cloned several times). Using an unique suffix (e.g. an UUID or counter with monotonous increment) for renaming each variable during cloning prevents variable collision. Moreover, a proper mechanism is required so that each module clone (as well as its ports) is identified by an unique URI.

3.2.5. Definition of a Module Operator

While connecting modules consists in performing a join operation and retrieving the intersection of their respective results, composing a query may require other operations such as union or negation. For example, to retrieve the names of the persons and the instruments they play, one would connect the personID port of the "Person" module to the personPlaying port of the "Person plays Instrument" module, and it would result in computing a join between the persons who have a name and the persons who play some instrument. However, to retrieve the name of the persons who do not play any instrument, one has to apply a negation operation on the "Person plays Instrument" module before connecting the result to the "Person" module (cf. left side of Figure 5C).

We define a *module operator* as a transformation of the meaning of one or several modules. This transformation creates a new module, therefore, it has to transform the modules' SPARQL fragments into another SPARQL fragment. Examples of module operators encompass setting some possible values for a port (corresponding to the VALUES pattern in SPARQL), declaring that a module is not necessarily present (OPTIONAL pattern), is absent (FILTER NOT EXISTS pattern), or performing the union (UNION pattern) or the difference (MINUS pattern) of two modules.

As module operators generate new modules, we model them as a subclass of modules. A module operator has one "parameter port" for each module the operation requires, and one "result port" that exposes the result of the operation. Parameter ports allow to connect the module operator to the module(s) one wants to perform the operation on. A parameter port can only be connected to at most one port. As parameter ports have specific characteristics, and the connection between an operator's parameter port and another module conveys a different meaning than the port connection seen in Section 3.2.3, we model them as a subclass of module port different from datatype ports and URI ports. Instead of a SPARQL variable, a parameter port is associated to a parameter name (each parameter of an operator has a different name), and this name appears at least once in the operator SPARQL fragment. For example, the absence operator has one parameter named `parameter0`, and the operator's SPARQL fragment would be

```
FILTER NOT EXISTS { parameter0 } .
```

When the operator parameter is connected to the module it should perform its operation on, 1) the variable of the module's port is renamed by the variable of the operator's result port, 2) the parameter name is replaced by the module's SPARQL fragment in the operator's SPARQL fragment, 3) the parameter port is removed from the set of ports of the operator, 4) the operator's result port is updated by information from the module's port (its syntactic and semantic types, whether it should appear in

the SELECT part of the query), and 5) all the other module's ports are added to the operator's set of ports. Note that when not all the parameter ports of an operator are connected, the operator's SPARQL fragment still contains these parameters names, so it is not syntactically-valid SPARQL. However, when all the parameter ports have been connected, each occurrence of their names have been replaced by the SPARQL fragment of a module, and the SPARQL fragment of the operator is now syntactically valid. Also note that the parameter ports and the result port of an operator are neither datatype ports nor URI ports when the operator is created because it is not known yet to which port or which module they will be connected. However, after being connected, the parameter ports are removed from the set of ports and the result port is updated by the port the parameter was connected to, so the operator's set of ports only contains datatype or URI ports. At this point, the operator eventually has a valid SPARQL fragment, its set of ports is the union of the ports of the module(s) its parameter(s) are connected to, and its result port is one of them, so the operator now has all the characteristics of a module as defined in Section 3.2.1.

The result port exposes the result of the operator on the parameter(s), so that it can in turn be connected to other modules, as shown in Figure 5C, or to a parameter port of another operator, which allows to have nested operations. Because of the process exposed in this section, the result port of an operator can only be connected to another module once all the operator parameters are connected (which is consistent with the fact that the operation can only be performed if all the parameters it depends on have a value). Because the result port has been updated by the port of the module the operator's parameter was connected to, it is now fully typed and proper type consistency check can be performed. For example in Figure 5C, observe that we connect the "person" port of Module1 to the result port of Module3, which has previously been updated by the "person" port of Module2.

In Section 3.3.3, we provide the SPARQL fragments of operators OPTIONAL, FILTER NOT EXISTS, UNION, MINUS, VALUES and SERVICE.

3.3. Map interactions with modules to operations on their SPARQL fragments

In this section, we first define how to generate a SPARQL query from a module's SPARQL fragment, and then how to map interactions with modules to operations on their SPARQL fragments (Figure 4). The SPARQL1.1 specification[4] defines how results are determined from SPARQL queries.

3.3.1. Generate the SPARQL query corresponding to a module

The SPARQL query corresponding to a module is generated by filling the module's set of prefix declarations, the variables of its ports selected for the projection (cf. Section 3.2.2), and its SPARQL fragment in the following template (cf. Figure 5A):

```
# INSERT PREFIXES DECLARATION HERE

SELECT # INSERT LIST OF THE VARIABLES OF SELECTED PORTS HERE
WHERE {
  # INSERT SPARQL FRAGMENT HERE
}
```

3.3.2. Connect the ports of modules

Connecting the port $P1$ of a module $M1$ to the port $P2$ of a module $M2$ generates a new module $M3$ (cf. Figure 5 B). It conveys the meaning that $P1$ and $P2$ refer to the same element, so they will be merged and the associated variable will be called $V3$. $P1$ and $P2$ are therefore assumed to be syntactically and semantically compatible. $V3$ can be the name of the variable of $P1$, or that of $P2$, or any value as long as it is guaranteed to be unique among all the modules.

[4] https://www.w3.org/TR/sparql11-query/

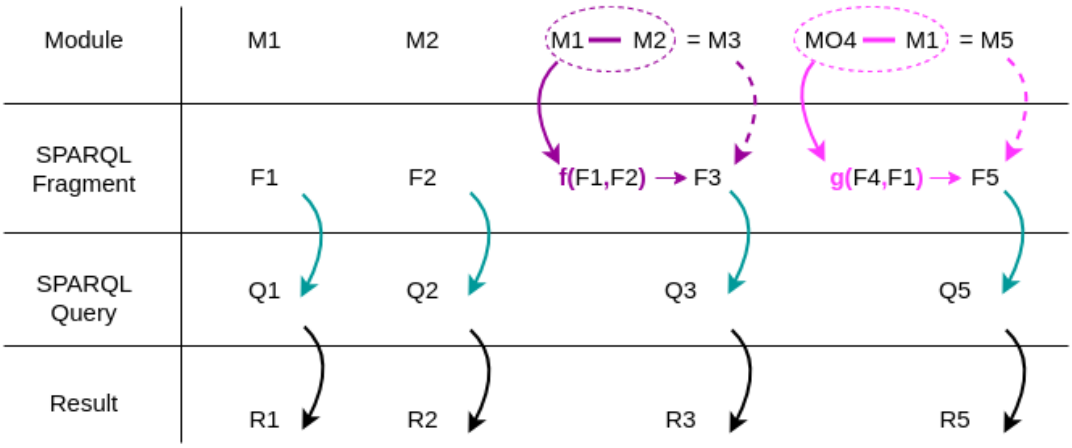

Figure 4: Mapping between interactions on modules and operations on their SPARQL fragments. A module corresponds to a concept over the information present in a knowledge base, and is composed of a SPARQL fragment. Mx represents a module, and MOx a module operator. The SPARQL fragment associated to Mx is noted Fx, the SPARQL query Qx, and the result Rx. Left: a SPARQL query can be generated from this fragment (turquoise arrow) and be sent to the knowledge base endpoint (black arrow), that retrieves the knowledge base elements matching the query. Middle: when two modules are connected through some of their port (purple connection), we define the operation ($f(...)$ corresponds to `connectPorts(...)` detailed in Section 3.3.2) that is performed on their SPARQL fragments (purple arrow) and that generates the SPARQL fragment of the resulting concept. Right: when a module operator is connected to a module's port (pink connection), we define the operation ($g(...)$ corresponds to `connectOperatorParameterPort(...)` detailed in Section 3.3.3) that is performed on their SPARQL fragments (pink arrow) and that generate the SPARQL fragment of the resulting concept.

The SPARQL fragment of $M3$ depends on whether $P1$ and $P2$ belong to the same module.

The function `renameVariable(port, module, newVar)` takes as parameters: a port, the module this port is part of, and a new name for the variable that is guaranteed to be unique among all the modules. This function replaces all the occurrences of the variable associated to `port` by `newVar` in the SPARQL fragment of `module`, and sets the variable name of `port` to `newVar`. It returns `module`.

If $M1$ and $M2$ are the same module:

$$\begin{aligned} connectPorts(P1, M1, P2, M1, V3) &= renameVariable(P1, M1', V3) \\ \text{where } M1' &= renameVariable(P2, M1, V3) \end{aligned} \tag{1}$$

The function `mergeModules(M1, M2)` returns a new module which set of ports is the union of those of $M1$ and $M2$, which set of prefixes is the union of those of $M1$ and $M2$, and which SPARQL fragment is the concatenation of those of $M1$ and $M2$.

If M1 and M2 are different modules:

$$\begin{aligned} connectPorts(P1, M1, P2, M2, V3) &= mergeModules(M1', M2') \\ \text{where } M1' &= renameVariable(P1, M1, V3) \\ \text{and } M2' &= renameVariable(P2, M2, V3) \end{aligned} \tag{2}$$

Note that if $P1$ is connected to more than one other ports, the order in which the connections are processed is irrelevant.

3.3.3. Connect an operator parameter port to a module port

Connecting the port $P2$ of a module $M2$ to the parameter port $P3$ of a module operator $MO3$ generates a new module $M4$ (cf. Figure 5 C). It conveys the meaning of performing the operation $MO3$ on the module $M2$, so $P3$ will be replaced by $M2$ before executing $MO3$. The result of the operation $MO3$ is exposed as the result port $R3$.

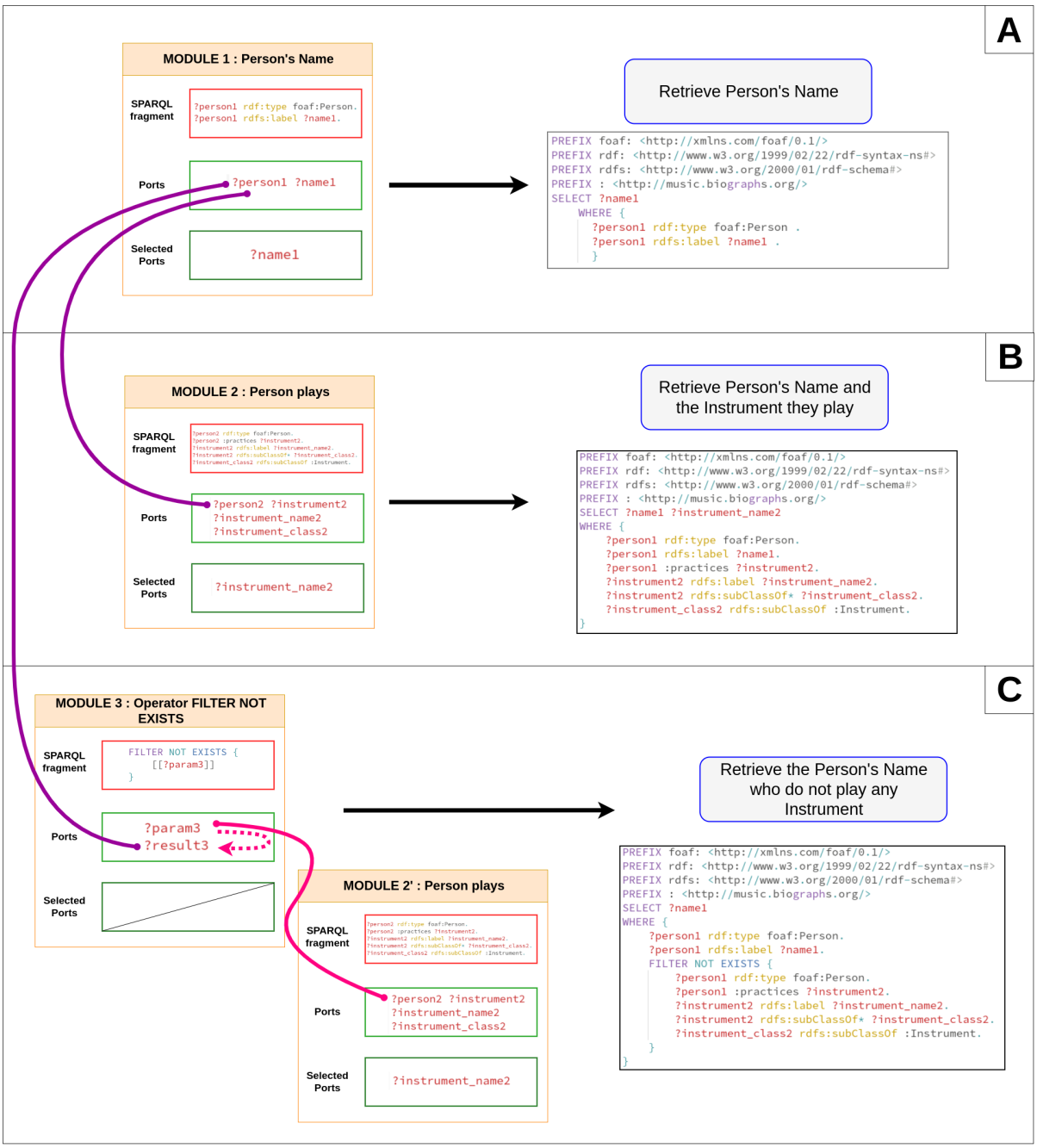

Figure 5: Generation of SPARQL code. (A) Case of a classical module. On the left, information of a given instance of module (Module1). On the right the corresponding resulting SPARQL query. This illustrates the association (blue arrow) in Figure 4 and its representation is given in Figure 6. **(B)** Case of a connection between two modules. On the left, the instantiation of Module 2 is connected to Module 1. On the right the SPARQL query resulting from the connection. **(C)** Case of an operation on modules. On the left, the different instantiations of modules and operator as well as the connections between them. On the right the corresponding resulting SPARQL query. This illustrates the association between operation and SPARQL in Figure 4 (purple arrow).

The function `fillOperatorParameter(paramPort, operatorModule, module)` takes as parameters: a module operator parameter port, the module operator the parameter port is part of, and a module). This function replaces all the occurrences of the parameter name in the operator's SPARQL fragment by the module's SPARQL fragment, add the module's set of prefix declarations to the operator's set of prefixes, and adds the module's set of ports to the operator's set of ports.

$$connectOperatorParameterPort(P3, R3, MO3, P2, M2) = fillOperatorParameter(P3, MO3, M2')$$
$$\text{where } M2' = renameVariable(P2, M2, R3) \tag{3}$$

Note that $P2$ is renamed by the operator's result port $R3$ before filling $P3$. For module operators with more than one parameter (e.g. UNION), this ensures that that fragments of each module connected to a parameter shares a variable with the others. The example, in order to retrieve the persons who know Alice or play violin, one would connect the first parameter port of an UNION operator to the `person1` port of the module "persons who know Alice", and the second parameter port to the `person2` port of the module "persons who play violin", and we need the SPARQL fragments of these two modules to use the same variable after being merged into the two blocks of UNION.

Also note that multiple operators can be nested by connecting the result port of the inner operator to the parameter port of the outer operator.

SPARQL fragment of operator OPTIONAL

```
OPTIONAL { parameter0 } .
```

SPARQL fragment of operator FILTER NOT EXISTS

```
FILTER NOT EXISTS { parameter0 } .
```

SPARQL fragment of operator MINUS

```
MINUS { parameter0 } .
```

SPARQL fragment of operator UNION

```
{
  parameter0
}
UNION
{
  parameter1
} .
```

SPARQL fragment of operator VALUES

```
VALUES ?result { listOfValues } .
```

Note that the operator VALUES does not have any parameter port, and requires the user provides a list of values. When the result port is connected to another port, the `?result` variable and the other port's variables are renamed, and the VALUES clause is concatenated to the other module's SPARQL fragment.

SPARQL fragment of operator SERVICE

```
SERVICE <serviceURL>
{
  parameter0
} .
```

Note that the operator SERVICE requires the user to provide an URL for the remote endpoint.

4. Implementation

4.1. A Python library to create modules, combine them, and generate the corresponding SPARQL queries

The `sparqlModule`[5] Python library provides an implementation allowing to create modules, combine them and generate the corresponding SPARQL queries. It also provides a collection of modules[6] and some examples[7] with Python and SPARQL code.

4.2. Module serialization

We propose an RDF serialization of modules (Figure 6). In addition to information about the structure of a module and its ports that is required for module creation, this can be complemented by other metadata such as keywords or usage statistics that could facilitate module discovery or dynamic identification of the modules that have a port compatible with a set of modules selected previously.

4.3. Module connections serialization

Section 3.3 presented how connected modules result on the creation of a more complex module. However, it may also be interesting to keep track of how this complex module was assembled from its components. We propose an RDF serialization of module composition (Figure 7) that can be used to extend the description of a module.

Note that if a module was used to generate a SPARQL query, the module description (including its optional composition) can be used to complement the query description (e.g. queries on the SIB endpoints[8] [12]).

5. Discussion and future directions

For the sake of simplicity, in this article as well as in the example section of the Python library, the modules we manipulated have short and simple SPARQL fragments. However, modules are more adequate when they encapsulate large or complex portions of SPARQL. Assessing this aspect is beyond the scope of this article though, and it is the subject of an ongoing study, which we detail briefly in Section 5.4.

5.1. Query performances

Query performance is an important factor for user adoption, as it directly affects both user experience and practical deployment. This raises the questions of 1) determining the optimal formulation of the SPARQL fragment of a module, and 2) whether queries composed of such "optimal" modules automatically have themselves an optimal SPARQL code.

These questions mandate a dedicated study. It should be noted that optimality here is challenging to define, as different SPARQL engines have different query processing strategies. Additionally, maybe these engines' strategies could be improved by leveraging the graph of connected modules. This sets the path for two approaches: determining the optimal order for processing module connections in order to generate an universal optimal query to be passed to the SPARQL engine, or sending the graph of modules to some module-aware SPARQL engine that would be able the leverage its internal indexing for determining the connection processing order and possibly some clever rewriting of the SPARQL fragments.

[5] https://gitlab.com/odameron/sparqlModulePython
[6] https://gitlab.com/odameron/sparqlModulePython/-/tree/main/moduleCollection
[7] https://gitlab.com/odameron/sparqlModulePython/-/tree/main/example
[8] https://github.com/sib-swiss/sparql-examples/tree/master

```
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix schema: <https://schema.org/> .
@prefix smodule: <https://sparql_module.biographs.org/> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix foaf: <http://xmlns.com/foaf/0.1/> .
@prefix : <http://music.biographs.org/> .
@prefix music_mod: <http://music_module.biographs.org/> .

music_mod:Module1 a smodule:SPARQLModule;
  rdfs:label "Information about Person"@en;
  rdfs:comment "Information about Person"@en;
  schema:keywords "Person";
  smodule:prefixes """PREFIX foaf: <http://xmlns.com/foaf/0.1/>
    PREFIX rdf: <http://www.w3.org/1999/02/22/rdf-syntax-ns#>
    PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
    PREFIX : <http://music.biographs.org/>
    """ ;
  smodule:fragment """
    ?person rdf:type foaf:Person .
    ?person rdfs:label ?name .
    ?person foaf:birthDate ?birth_date .
    """;
  smodule:output music_mod:Module1_port-person, music_mod:Module1_port-name,
        music_mod:Module1_port-birth_date.

music_mod:Module1_port-person
  a smodule:ModulePortURI;
  rdfs:domain foaf:Person ;
  smodule:variableName "person" ;
  rdfs:label "Person";
  smodule:isSelected "true"^^xsd:boolean .

music_mod:Module1_port-name
  a smodule:ModulePortDatatype;
  rdfs:domain xsd:string ;
  smodule:variableName "name" ;
  rdfs:label "Name" :
  smodule:isSelected "true"^^xsd:boolean .

music_mod:Module1_port-birth_date
  a smodule:ModulePortDatatype;
  rdfs:domain xsd:date;
  smodule:name "birth_date"
  rdfs:label "Birthdate" ;
  smodule:isSelected "false"^^xsd:boolean.
```

Prefixes

Metadata

Prefixes for the SPARQL fragment

SPARQL fragment

Ports

Description of each Port

Figure 6: Representation of a Module with RDF The module represented here corresponds to 'Person's Name and birth date'.

One could observe that the responsibility of designing a sensible-if-not-optimal SPARQL fragment for a module is on the knowledge engineers' who design the modules, and not on the module framework itself. This task requires SPARQL proficiency, a good knowledge of the endpoint data schema, as well as some experience of the endpoint engine specificity. Similarly to building ontologies, a set of guidelines would certainly be valuable, as we may not know (yet) how to write an optimal SPARQL fragment, but have a pretty good experience on how not to write a bad one.

In addition to the theoretical study mentioned above, we are currently conducting a practical evalua-

```
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix dcterms: <http://purl.org/dc/terms/> .
@prefix : <https://sparql_module.biographs.org/> .
@prefix music_mod: <http://music_module.biographs.org/> .

:module_composition_001 dcterms:title "Name of the persons who know Alice and
        the instruments they play".
:module_composition_001 rdfs:comment "Find all the persons who know Alice and
        play one or more Instrument(s). Display their name and the
        instrument(s) they play".
:module_composition_001 dcterms:subject "Instrument", "Knows Alice".
:module_composition_001 dcterms:creator "Yael Tirlet".

:module_composition_001 rdf:type :SPARQL_Module_Composition.
:module_composition_001 :hasConnection :c1, :c2, :c3.

:c1 rdf:type :Connection.
:c1 :connects music_mod:Module1_port-person, music_mod:Module2_port-person.

:c2 rdf:type :Connection.
:c2 :connects music_mod:Module1_port-person, music_mod:Module3_port-person1.

:c3 rdf:type :Connection.
:c3 :connects music_mod:Module3_port-person2, music_mod:Module4_port-person.
```

Prefixes

Metadata

Module Composition : Connections

For each Connection: the URIs of the two connected Ports

Figure 7: Description of the module composition with RDF The description corresponds to the module composition illustrated in Figure 1. The composition is described by its connections, each of them referring to a particular port by its URI.

tion of the performances of the queries generated from modules on the UniProtKB compared to the reference queries.

5.2. Modules as a functional view complementing the structure of the data schema

Modules provide a functional view of a knowledge base, whereas the data schema provides a technical view. It should be noted that the decomposition of a knowledge base data schema into modules may not be unique. This raises the question of the balance between many low-level and simple modules that offer a great flexibility, and fewer higher level and possibly overlapping modules. This balance is likely to depend from both the knowledge base and from the types of queries users are interested in, so in addition, there may be several set of modules over a knowledge base that address different use cases.

We assume that designing a collection of modules for a knowledge base is the responsibility of knowledge engineers who are familiar with the RDF data schema of the endpoint. To what extent this task can be automated or assisted remains to be studied.

5.3. User interface

In this article, we have covered the principles of module composition. The Python library provides the operational framework. It supports the creation of a collection of serialized modules by knowledge engineers, but we acknowledge that our target user base is not likely to reuse them as such. We are currently working on a graphical interface (cf. infra) on top of the Python library that would support visual interactions similar to the diagrams from Figure 5 or from the examples of the Python library.

For any knowledge base exposed as a SPARQL endpoint, maintainers provide some documentation in the form of diagrams illustrating the data schema, and a collection of examples of representative SPARQL queries. Maintainers could also provide a set of module templates, serialized as in Section 4.2. A graph which vertices are the modules, and edges indicate that two modules can be connected would

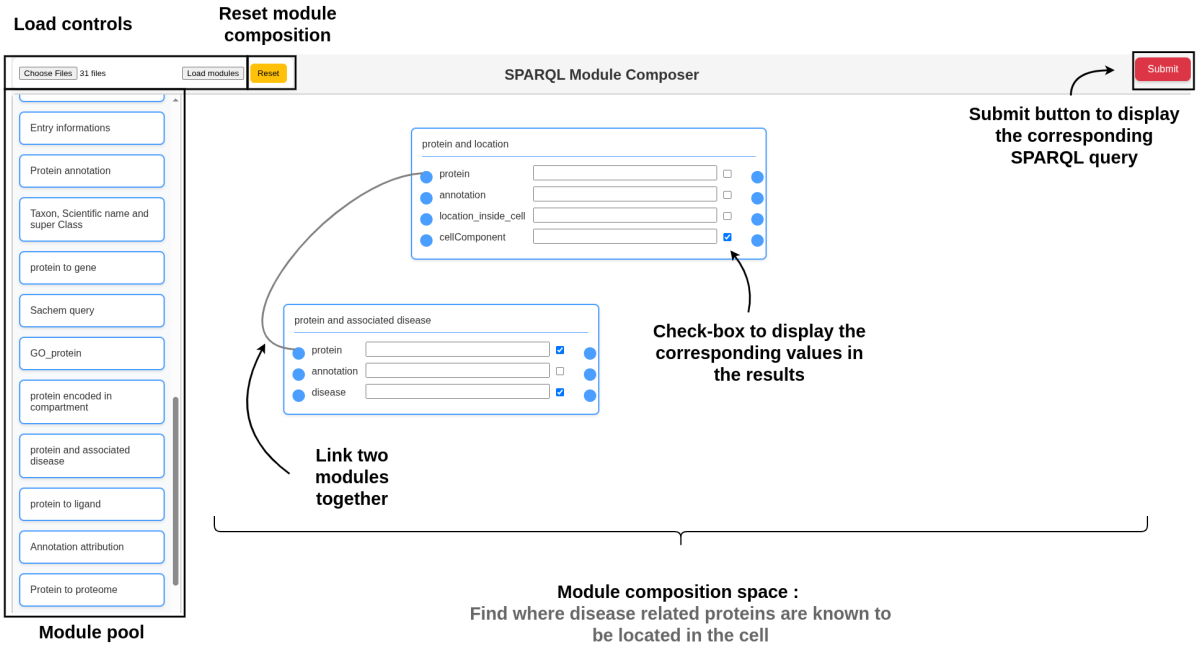

Figure 8: Graphical interface of the Web application allowing to compose queries on the UniProtKB public SPARQL endpoint by connecting modules. Composition of the UniProtKB reference query n°21: "Find where disease related proteins are known to be located in the cell". Module templates are displayed in the left pane (with ports hidden for saving real estate). They can be loaded from files such as in Figure 6. Dragging a module template into the composition pane generates a clone, with distinct identifiers and variable names (and ports visible). Module ports can be connected by clicking on a port, and releasing the click on the other port. The submit button generates the SPARQL query. The connections between the modules can also be saved in a file similar to Figure 7. The code for the modules, the resulting SPARQL query and a pointer to the reference query are available in the example section of the Python library (Section 4.1).

materialize the functional view as a counterpart to the data schema diagrams.

Based on this collection of module templates, the user interface would allow to 1) create modules by cloning (cf. Section 3.2.4), 2) represent each module as a box having a set of port, and 3) visually compose a query by interacting with these modules and connecting them, while enforcing the syntactic and semantic consistency of these connections. The user interface would then rely on the Python library to generate the corresponding SPARQL code, send it to the endpoint and retrieve the results.

While we anticipate that the set of modules provides a simpler view than the raw data schema, 1) this remains to be studied, and 2) although simpler, this view may still remain overwhelming. To address this potential issue, we can take advantage of the semantic consistency constraints: when the user starts assembling modules, we can highlight the candidate modules that are compatible with the ones currently used, or re-rank the candidate modules that have been frequently connected with them in a collection of queries.

5.4. Application to complex knowledge base: UniProtKB use-case

In an ongoing study, we designed 31 modules based on the descriptions of a selection of 103 SPARQL queries from the set of reference SPARQL queries on UniProtKB[9] [12]. We showed that different combinations of these module allow to reconstruct 56 of the 103 original queries with the operators already implemented. 14 additional queries are feasible with our modules but contain some COUNT, GROUP BY or ORDER BY in the query. Another 20 queries are feasible excepting the FILTER or BIND part that we do not support yet (these operators would require users to modify the SPARQL fragments of a module). 7 of the reference queries contain some OPTIONAL clause in places where our modules

[9] https://github.com/sib-swiss/sparql-examples

had none. The 6 remaining queries each contain a triple that is not present in any other query.

As a proof of concept, we designed an ad hoc visual interface (Figure 8). We are currently refining the GUI, and will work on providing a generic mechanism that will be applicable to any knowledge base endpoint.

5.5. Endpoints and federated queries

Note that a module's SPARQL fragment can contain a SERVICE pattern (either because it was directly included into the fragment, or as a result of having a SERVICE operator as one of the module's components), so our approach is compatible with federated queries.

We decided not to associate a particular endpoint to each module. This allows modules to be applicable to any endpoint, as long as the module's SPARQL fragment is compatible with the endpoint data schema. For example, we contemplate the creation of a library of BioPAX-based modules that could be applicable to any BioPAX-compliant endpoint, such as Reactome of PathwayCommons.

5.6. Position with respect to LLMs

Chatbots are an alternative approach for letting users to articulate what they want to do and automatically generating the SPARQL code that represents how to do it [24]. However, this requires a learning set of queries on the same data schema, and the risk of approximation or hallucination remains. It would be interesting to study whether the textual description associated to each modules would provide useful material for a RAG-like approach.

Conversely, it would be interesting to investigate verbalization strategies for automatically generating the textual description of a module resulting from the connection of several modules.

6. Conclusion

We have proposed a framework that allows to represent a functional view on a SPARQL endpoint, and to map it with the technical view provided by the data schema. This framework relies on the notion of module, that represent a concept that makes sense for domain experts, and that is associated with a fragment of SPARQL code. Modules can be iteratively connected to create more complex modules, until the corresponding concept corresponds to what the user is looking for. Our framework can then generate the corresponding SPARQL query to be sent to the endpoint. This allows experts to focus on what they are looking for, while our system takes care of how to obtain it.

In ongoing or future works, we will further standardize the RDF representation of SPARQL modules for re-use as named components in SPARQL queries. The goal is to allow both direct SPARQL writers as well as GUI users to use SPARQL modules. In parallel, we plan on designing and sharing a collection of modules for common tasks independent of the knowledge base, such as retrieve all the ancestors of a class, all the descendants of a class, all the instances of a class or its descendants. Eventually, we plan to assess the added values of modules on complex knowledge bases such as UniProtKB or wikidata.

Acknowledgments

This work was supported by the French National Research Agency, project Endovire ANR-22-CE20-0005. We are thankful to the GenOuest bioinformatics platform for technical support. Maxime Léger and Alix Régnier performed preliminary work. We thank Hélène Jaudoin for insightful discussions.

Jerven Bolleman is a member of The Swiss-Prot group, a part of the SIB Swiss Institute of Bioinformatics and of the UniProt Consortium. Swiss-Prot group activities are supported by the Swiss Federal Government through the State Secretariat for Education, Research and Innovation SERI and by the UniProt is supported by the National Human Genome Research Institute (NHGRI), Office of Director [OD/DPCPSI/ODSS]; National Institute of Allergy and Infectious Diseases (NIAID), National Institute on Aging (NIA), National Institute of General Medical Sciences (NIGMS), National Institute of Diabetes

and Digestive and Kidney Diseases (NIDDK), National Eye Institute (NEI), National Cancer Institute (NCI),National Heart, Lung, and Blood Institute (NHLBI) of the National Institutes of Health (NIH) [U24HG007822];

Conflict of interest

The authors have no conflict of interest to disclose regarding this study.

Declaration on Generative AI

The author(s) have not employed any Generative AI tools.

References

[1] M. R. Kamdar, J. D. Fernández, A. Polleres, T. Tudorache, M. A. Musen, Enabling web-scale data integration in biomedicine through linked open data, NPJ digital medicine 2 (2019) 90. doi:https://doi.org/10.1038/s41746-019-0162-5.

[2] M. R. Kamdar, M. A. Musen, An empirical meta-analysis of the life sciences linked open data on the web, Scientific data 8 (2021) 24. doi:https://doi.org/10.1038/s41597-021-00797-y.

[3] S. Kawashima, T. Katayama, H. Hatanaka, T. Kushida, T. Takagi, Nbdc rdf portal: a comprehensive repository for semantic data in life sciences, Database : the journal of biological databases and curation 2018 (2018). doi:https://doi.org/10.1093/database/bay123.

[4] S. S. I. of Bioinformatics RDF Group Members, The sib swiss institute of bioinformatics semantic web of data, Nucleic acids research 52 (2024) D44–D51. doi:https://doi.org/10.1093/nar/gkad902.

[5] T. J. Callahan, I. J. Tripodi, A. L. Stefanski, L. Cappelletti, S. B. Taneja, J. M. Wyrwa, E. Casiraghi, N. A. Matentzoglu, J. Reese, J. C. Silverstein, C. T. Hoyt, R. D. Boyce, S. A. Malec, D. R. Unni, M. P. Joachimiak, P. N. Robinson, C. J. Mungall, E. Cavalleri, T. Fontana, G. Valentini, M. Mesiti, L. A. Gillenwater, B. Santangelo, N. A. Vasilevsky, R. Hoehndorf, T. D. Bennett, P. B. Ryan, G. Hripcsak, M. G. Kahn, M. Bada, W. A. Baumgartner, L. E. Hunter, An open source knowledge graph ecosystem for the life sciences, Scientific data 11 (2024) 363. doi:https://doi.org/10.5281/zenodo.7035867.

[6] T. Berners-Lee, J. Hendler, O. Lassila, The semantic web, Scientific American 284 (2001) 34–43. http://www.sciam.com/2001/0501issue/0501berners-lee.html.

[7] C. Bizer, T. Heath, T. Berners Lee, Linked data–the story so far, International Journal on Semantic Web and Information Systems 5 (2009) 1–22.

[8] S. Mukherjea, Information retrieval and knowledge discovery utilising a biomedical semantic web, Briefings in bioinformatics 6 (2005) 252–262.

[9] J. Pérez, M. Arenas, C. Gutierrez, Semantics and complexity of sparql, ACM Trans. Database Syst. 34 (2009) 16:1–16:45. doi:10.1145/1567274.1567278.

[10] S. Bechhofer, I. Buchan, D. De Roure, P. Missier, J. Ainsworth, J. Bhagat, P. Couch, D. Cruickshank, M. Delderfield, I. Dunlop, M. Gamble, D. Michaelides, S. Owen, D. Newman, S. Sufi, C. Goble, Why linked data is not enough for scientists, Future Generation Computer Systems 29 (2013) 599–611.

[11] T. Mendes de Farias, J. Wollbrett, M. Robinson-Rechavi, F. Bastian, Lessons learned to boost a bioinformatics knowledge base reusability, the bgee experience, GigaScience 12 (2022). doi:https://doi.org/10.1093/gigascience/giad058, in press.

[12] J. Bolleman, V. Emonet, A. Altenhoff, A. Bairoch, M.-C. Blatter, A. Bridge, S. Duvaud, E. Gasteiger, D. Kuznetsov, S. Moretti, P.-A. Michel, A. Morgat, M. Pagni, N. Redaschi, M. Zahn-Zabal, T. M. de Farias, A. C. Sima, A large collection of bioinformatics question–query pairs over federated knowledge graphs: methodology and applications, GigaScience 14 (2025) giaf045. doi:10.1093/gigascience/giaf045.

[13] J. C. van Dam, J. J. Koehorst, P. J. Schaap, V. A. Martins Dos Santos, M. Suarez-Diez, Rdf2graph a tool to recover, understand and validate the ontology of an rdf resource, Journal of biomedical semantics 6 (2015) 39.

[14] L. McCarthy, B. Vandervalk, M. Wilkinson, Sparql assist language-neutral query composer, BMC bioinformatics 13 Suppl 1 (2012) S2.

[15] B. Broll, A. Lédeczi, P. Volgyesi, J. Sallai, M. Maroti, A. Carrillo, S. L. Weeden-Wright, C. Vanags, J. D. Swartz, M. Lu, A Visual Programming Environment for Learning Distributed Programming, in: Proceedings of the 2017 ACM SIGCSE Technical Symposium on Computer Science Education, SIGCSE '17, Association for Computing Machinery, New York, NY, USA, 2017, pp. 81–86. doi:10.1145/3017680.3017741.

[16] M. Resnick, J. Maloney, A. Monroy-Hernández, N. Rusk, E. Eastmond, K. Brennan, A. Millner, E. Rosenbaum, J. Silver, B. Silverman, Y. Kafai, Scratch: programming for all, Communications of the ACM 52 (2009) 60–67. doi:10.1145/1592761.1592779.

[17] P. Bottoni, M. Ceriani, Linked Data Queries as Jigsaw Puzzles: a Visual Interface for SPARQL Based on Blockly Library, in: Proceedings of the 11th Biannual Conference of the Italian SIGCHI Chapter, CHItaly '15, Association for Computing Machinery, New York, NY, USA, 2015, pp. 86–89. doi:10.1145/2808435.2808467.

[18] D. Schweiger, Z. Trajanoski, S. Pabinger, Sparqlgraph: a web-based platform for graphically querying biological semantic web databases, BMC bioinformatics 15 (2014) 279. doi:https://doi.org/10.1186/1471-2105-15-279.

[19] A. Yamaguchi, K. Kozaki, K. Lenz, H. Wu, N. Kobayashi, An intelligent sparql query builder for exploration of various life-science databases, in: Proc. of the 3rd International Conference on Intelligent Exploration of Semantic Data, volume 1279, 2014, pp. 83–94.

[20] T. Francart, Sparnatural: A Visual Knowledge Graph Exploration Tool, in: C. Pesquita, H. Skaf-Molli, V. Efthymiou, S. Kirrane, A. Ngonga, D. Collarana, R. Cerqueira, M. Alam, C. Trojahn, S. Hertling (Eds.), The Semantic Web: ESWC 2023 Satellite Events, volume 13998, Springer Nature Switzerland, Cham, 2023, pp. 11–15. doi:10.1007/978-3-031-43458-7_2, series Title: Lecture Notes in Computer Science.

[21] H. Chiba, I. Uchiyama, Spang: a sparql client supporting generation and reuse of queries for distributed rdf databases, BMC bioinformatics 18 (2017) 93.

[22] S. Campinas, T. E. Perry, D. Ceccarelli, D. Renaud, G. Tummarello, Introducing rdf graph summary with application to assisted sparql formulation, in: 23rd International Workshop on Database and Expert Systems Applications, IEEE, 2012, pp. 261–266.

[23] M. Djokic-Petrovic, V. Cvjetkovic, J. Yang, M. Zivanovic, D. J. Wild, PIBAS FedSPARQL: a web-based platform for integration and exploration of bioinformatics datasets, Journal of biomedical semantics 8 (2017) 42.

[24] V. Emonet, J. Bolleman, S. Duvaud, T. M. d. Farias, A. C. Sima, LLM-based SPARQL Query Generation from Natural Language over Federated Knowledge Graphs, 2024. URL: http://arxiv.org/abs/2410.06062, arXiv:2410.06062 [cs].